

```
# Compute squares using a loop
for i in range(1, 11):
    print(f"{i}^2 = {i*i}")
```

```
1^2 = 1
2^2 = 4
3^2 = 9
4^2 = 16
5^2 = 25
6^2 = 36
7^2 = 49
8^2 = 64
9^2 = 81
10^2 = 100
```

```
# Define a function for factorial
def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

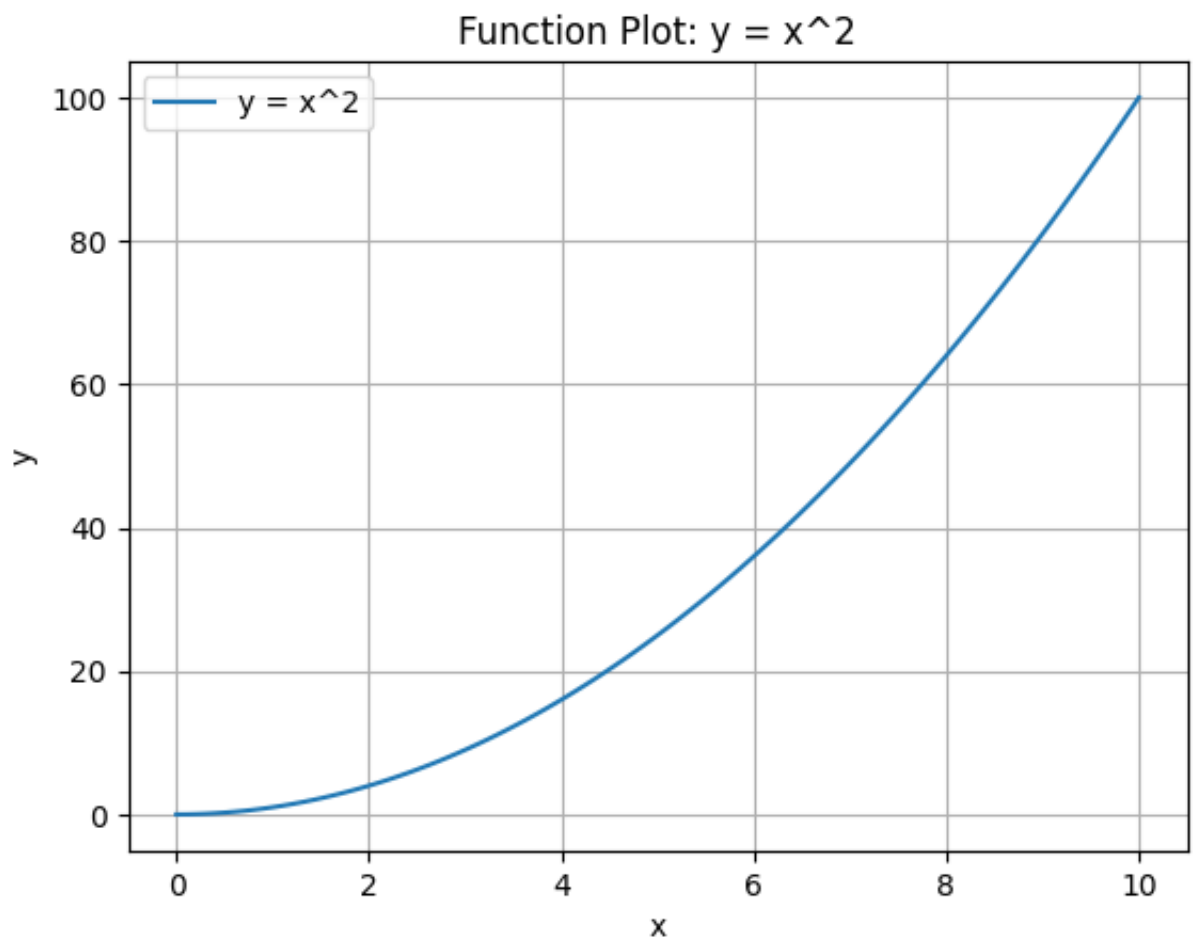
# Test the factorial function
print("Factorial of 5:", factorial(5))
```

```
Factorial of 5: 120
```

```
import numpy as np
# Create an array
arr = np.arange(1, 11)
# Compute the square of each element
squared = arr**2
print("Squares:", squared)
# Compute the sum of elements
total_sum = np.sum(arr)
print("Sum of array elements:", total_sum)
```

```
Squares: [ 1  4  9 16 25 36 49 64 81 100]
Sum of array elements: 55
```

```
import matplotlib.pyplot as plt
# Define the function
x = np.linspace(0, 10, 100)
y = x**2
# Plot the function
plt.plot(x, y, label="y = x^2")
plt.title("Function Plot: y = x^2")
plt.xlabel("x")
plt.ylabel("y")
plt.grid()
plt.legend()
plt.show()
```



```
x = 1 / 49
print(x)
```

```
0.02040816326530612
```

```
x = x * 49
print(x)
```

```
0.9999999999999999
```

CASE 1

```
def true_error(true_value, approx_value):
    return abs(true_value - approx_value)

def true_relative_error(true_value, approx_value):
    return abs(true_value - approx_value) / abs(true_value)
```

```
x = 1 / 49
approx = x * 49
true_value = 1.0
```

```
TE = true_error(true_value, approx)
TRE = true_relative_error(true_value, approx)
```

```
print("CASE 1")
print(" true value      =", true_value)
print(" approx value    =", approx)
print(" true error      =", TE)
print(" true rel error   =", TRE)
```

CASE 1

```
true value      = 1.0
approx value     = 0.9999999999999999
true error       = 1.1102230246251565e-16
true rel error   = 1.1102230246251565e-16
```

```
a = 0.1
b = a + a + a
print(b == 0.3)
```

```
False
```

```
# CASE 2
def true_error(true_value, approx_value):
    return abs(true_value - approx_value)

def true_relative_error(true_value, approx_value):
    return abs(true_value - approx_value) / abs(true_value)

a = 0.1
approx = a + a + a
true_value = 0.3

TE = true_error(true_value, approx)
TRE = true_relative_error(true_value, approx)

print("CASE 2")
print(" true value      =", true_value)
print(" approx value    =", approx)
print(" true error       =", TE)
print(" true rel error    =", TRE)
```

```
CASE 2
true value      = 0.3
approx value    = 0.30000000000000004
true error      = 5.551115123125783e-17
true rel error   = 1.8503717077085943e-16
```

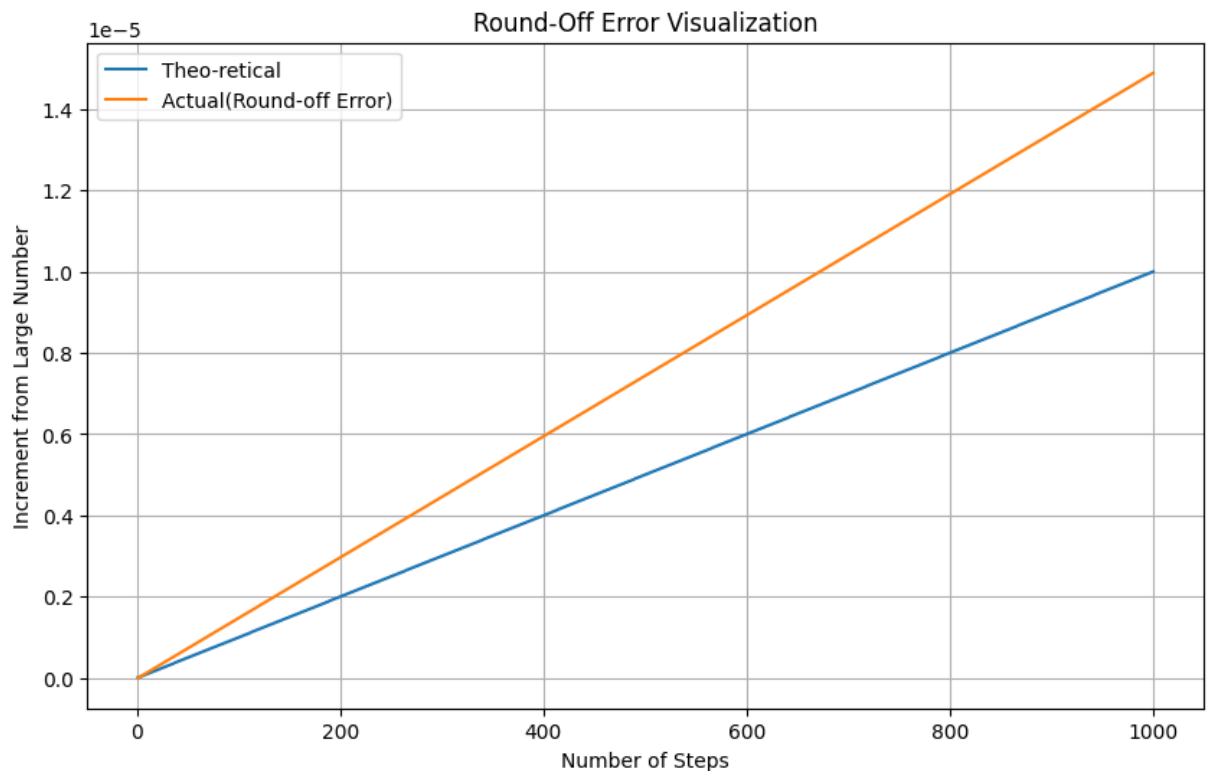
```
import numpy as np
import matplotlib.pyplot as plt
# Parameters
large_number = 1e8
small_increment = 1e-8
n_steps = 1000
# Compute results
theoretical = large_number + np.arange(1, n_steps + 1) * small_incr
actual = np.full(n_steps, large_number)

for i in range(1, n_steps):
    actual[i] = actual[i - 1] + small_increment
# Plot results
plt.figure(figsize=(10, 6))

plt.plot(range(1, n_steps + 1), theoretical - large_number, label="")

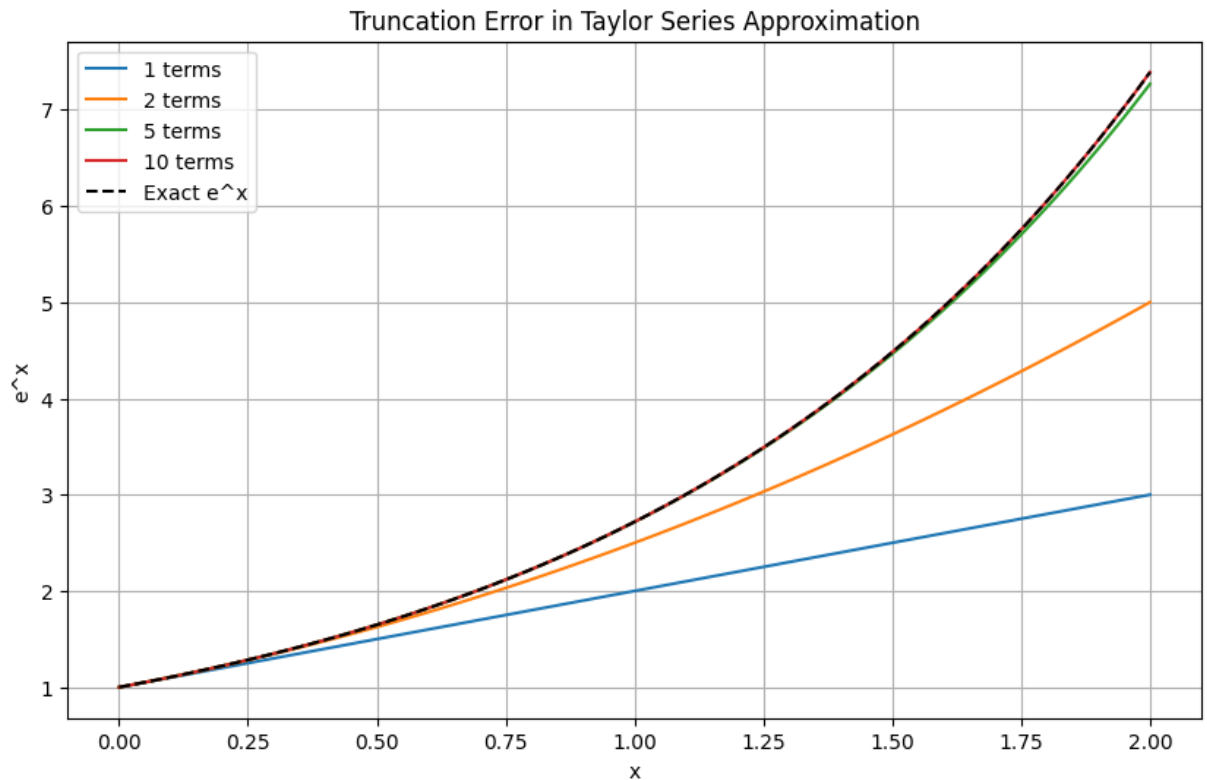
plt.plot(range(1, n_steps + 1), actual - large_number, label="Actual")
plt.title("Round-Off Error Visualization")
plt.xlabel("Number of Steps")
```

```
plt.ylabel("Increment from Large Number")
plt.legend()
plt.grid()
plt.show()
```



```
# Function to compute Taylor series approximation
def taylor_exp(x, n_terms):
    approximation = np.zeros_like(x)
    for n in range(n_terms + 1):
        approximation += x**n / factorial(n)
    return approximation
# Parameters
x = np.linspace(0, 2, 100) # x values to approximate e^x
exact = np.exp(x) # Exact values
n_terms_list = [1, 2, 5, 10] # Different numbers of terms
# Plot results
plt.figure(figsize=(10, 6))
for n_terms in n_terms_list:
    approx = taylor_exp(x, n_terms)
    plt.plot(x, approx, label=f"{n_terms} terms")
plt.plot(x, exact, "k--", label="Exact e^x")
plt.title("Truncation Error in Taylor Series Approximation")
plt.xlabel("x")
```

```
plt.ylabel("e^x")
plt.legend()
plt.grid()
plt.show()
```



CASE 3

```
import numpy as np

def factorial(n):
    result = 1
    for i in range(1, n + 1):
        result *= i
    return result

def taylor_exp(x, n_terms):
    approx = np.zeros_like(x)
    for n in range(n_terms + 1):
        approx += x**n / factorial(n)
    return approx

def true_error_vec(true, approx):
    return np.abs(true - approx)
```

```
def true_relative_error_vec(true, approx):
    return np.abs(true - approx) / np.abs(true)

x = np.linspace(0, 2, 100)
true_values = np.exp(x)

n_terms_list = [1, 2, 5, 10]

print("CASE 3")
for n_terms in n_terms_list:
    approx = taylor_exp(x, n_terms)
    TE = true_error_vec(true_values, approx)
    TRE = true_relative_error_vec(true_values, approx)

    print(f"\nn_terms = {n_terms}")
    print(" max true error      =", np.max(TE))
    print(" max true rel error   =", np.max(TRE))
```

CASE 3

```
n_terms = 1
max true error      = 4.38905609893065
max true rel error  = 0.5939941502901619

n_terms = 2
max true error      = 2.3890560989306504
max true rel error  = 0.32332358381693654

n_terms = 5
max true error      = 0.1223894322639838
max true rel error  = 0.016563608480614472

n_terms = 10
max true error      = 6.138993594273501e-05
max true rel error  = 8.308224368687552e-06
```